

Request IDs, Correlation & Collision, Worked Out by Hand

Why the JSON-RPC `id` field is the one thing that makes concurrent MCP traffic possible — counted exactly, with the collision math derived.

What this document does. JSON-RPC allows many requests to be “in flight” at once on a single connection, and responses may come back in *any* order. The `id` field is what re-pairs each response with the request that asked for it. This document counts exactly how badly things break without it ($k!$ possible mis-pairings), derives the probability that randomly chosen IDs collide (the birthday bound), and shows why a monotonic counter makes the problem disappear. Every number is reproducible from Appendix A.

Concepts covered: in-flight multiplexing · the pending map · permutations $k!$ · the birthday collision bound · monotonic counters and the 2^{53} JSON-safe range · notifications (the `id-less` message).

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: three requests in flight on one wire

A client opens one connection to a server and fires three requests without waiting for any of them to return — this is the whole reason JSON-RPC was chosen for MCP (“multiple concurrent in-flight requests”). Each carries an integer `id`:

id	method	typical latency
7	<code>tools/call</code> (a slow query)	~ 900 ms
8	<code>resources/read</code> (a small file)	~ 40 ms
9	<code>ping</code>	~ 5 ms

Because each method takes a different time, the responses come back in the order 9,8,7 — the *reverse* of how they were sent. The transport delivers a flat stream of three response frames; nothing about their arrival order tells you which request each belongs to. Only the echoed `id` does.

Intuition

The `id` is a primary key. The client keeps a “pending map” $\{\text{id} \mapsto \text{waiting caller}\}$; when a response frame arrives it reads the `id`, looks up the caller in $O(1)$, hands over the **result**, and deletes the entry. Without that key, a connection could carry exactly *one* request at a time — send, block, receive, repeat — and all concurrency would be gone.

2 Step 1 — How bad is “no id”? Count the pairings

Suppose the `id` field did not exist. The client sent k requests and now k responses come back in some order. To use them it must guess which response answers which request — a bijection between k responses and k requests. The number of such bijections is

$$k! = k \cdot (k-1) \cdots 2 \cdot 1,$$

and exactly *one* of them is correct. For our $k = 3$:

$$3! = 6 \quad \text{orderings}, \quad P(\text{guess all correct}) = \frac{1}{3!} = \frac{1}{6} \approx 16.7\%.$$

Mini-example — this operation in isolation

List the $3! = 6$ ways to assign responses $\{r_a, r_b, r_c\}$ (arrival order) to requests $\{7, 8, 9\}$:

$$(7, 8, 9) \quad (7, 9, 8) \quad (8, 7, 9) \quad (8, 9, 7) \quad (9, 7, 8) \quad (9, 8, 7)$$

Only one matches the truth $7 \rightarrow r_c, 8 \rightarrow r_b, 9 \rightarrow r_a$. The other five silently hand a **ping** reply to a caller expecting query rows.

Watch out

$k!$ grows faster than any polynomial: $5! = 120$, $10! = 3,628,800$. The chance of a correct blind guess is not just small, it is *vanishing* as concurrency rises. “Just match them in order” only works if responses never reorder — and the entire point of async tool calls is that they do.

3 Step 2 — The rule that prevents collisions

The `id` solves Step 1 only if it is **unique among the requests currently in flight**. If the client reuses `id 7` while the first request 7 is still pending, a returning `id 7` is ambiguous — two callers claim it. That is an *id collision*, and it corrupts the pending map.

Two strategies are used in the wild. We analyse both with numbers.

3.1 Strategy A — monotonic counter (collisions impossible)

Allocate `id = c`, then $c \leftarrow c + 1$. Every request gets a fresh, never-before-used integer, so within a connection collisions cannot occur by construction. The only ceiling is JSON’s safe-integer range. JSON numbers are IEEE-754 doubles, exact up to

$$2^{53} = 9,007,199,254,740,992 \approx 9.0 \times 10^{15}.$$

At a sustained 1,000 requests/second, the time to exhaust that range is

$$\frac{2^{53}}{1000 \text{ req/s}} \approx 9.0 \times 10^{12} \text{ s} \approx 285,000 \text{ years}.$$

So overflow is a non-issue; the monotonic counter is the safe default.

3.2 Strategy B — random ids (the birthday bound)

Some clients pick random IDs from a space of size N . Now collisions are *possible*, and the probability follows the birthday problem. With n distinct draws already made, the chance all n are unique is

$$P(\text{no collision}) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right), \quad P(\text{collision}) = 1 - \prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right).$$

Using $1 - x \leq e^{-x}$ gives the clean upper-bound approximation

$$P(\text{collision}) \approx 1 - e^{-n(n-1)/(2N)} \approx \frac{n^2}{2N} \quad (\text{for } n \ll \sqrt{N}).$$

The famous consequence: collisions become likely around $n \approx 1.177\sqrt{N}$ (the point where $P = \frac{1}{2}$).

Mini-example — birthdays, the original case

Set $N = 365$ (days) and $n = 23$ people. The bound gives

$$P \approx 1 - e^{-23 \cdot 22 / (2 \cdot 365)} = 1 - e^{-0.693} = 1 - 0.500 = 50.0\%.$$

23 “requests” into a 365-slot ID space, a clash is already a coin flip — which is exactly why small or poorly-seeded random ID spaces are dangerous.

Concretely, for a server that tags requests with a random 16-bit id ($N = 2^{16} = 65,536$), the n giving a 50% chance of *some* collision is

$$n \approx 1.177 \sqrt{65,536} = 1.177 \times 256 \approx 301 \text{ requests.}$$

A busy session blows past 301 requests in seconds. A monotonic counter would have had zero risk over the same traffic.

Intuition

Why $\sqrt{N}$ and not N ? Collisions are about *pairs*. With n ids there are $\binom{n}{2} \approx n^2/2$ pairs, each colliding with probability $1/N$, so the expected number of clashes is $\approx n^2/2N$ — it crosses 1 when $n \approx \sqrt{N}$, long before you have used up the space. This is why “the space is huge” is a weak defence and “the ids are unique by construction” is a strong one.

4 Step 3 — The id-less message: notifications

JSON-RPC has a fourth shape with *no* id: the notification (method, no id, no response). That is deliberate — a notification is fire-and-forget, so there is nothing to correlate. The discriminator is purely structural:

message	id?	method?	result/error?
Request	✓	✓	—
Success response	✓	—	result
Error response	✓	—	error
Notification	—	✓	—

Watch out

Because a notification has no id, you must *not* wait for a reply to one — e.g. `notifications/initialized`. A pending-map entry that never resolves is a leaked future and a hang. “Has an id” is exactly “expects a response”; the two are the same bit.

5 The argument, at a glance

#	Idea	Formula	Value
1	Blind mis-pairings of k responses	$k!$	$3! = 6$
1	Chance of a correct blind guess	$1/k!$	16.7%
2	Monotonic counter ceiling	2^{53}	9.0×10^{15}
2	Years to exhaust at 1k req/s	$2^{53}/(10^3 \cdot \text{yr})$	$\sim 285,000$
2	Random-id collision (birthday)	$1 - e^{-n^2/2N}$	—
2	50% collision threshold	$n \approx 1.177\sqrt{N}$	301 at $N=2^{16}$

6 Equation cheat-sheet

Blind response-request pairings	$k!$
Correct-guess probability	$1/k!$
No-collision probability (n random ids)	$\prod_{i=0}^{n-1} (1 - i/N)$
Collision probability (bound)	$1 - e^{-n(n-1)/2N} \approx n^2/2N$
Birthday threshold ($P = \frac{1}{2}$)	$n \approx 1.177\sqrt{N}$
JSON safe-integer ceiling	$2^{53} = 9.007 \times 10^{15}$
Pending-map lookup cost	$O(1)$ per response

A Reference implementation

```
import math

def correct_guess_prob(k):
    return 1.0 / math.factorial(k)          # 1/k!

def collision_prob(n, N):
    p_unique = 1.0                          # exact birthday
    for i in range(n):
        p_unique *= (N - i) / N
    return 1.0 - p_unique

def half_threshold(N):
    return 1.177 * math.sqrt(N)             # n giving P 0.5

print(correct_guess_prob(3))                # 0.1666...
print(round(collision_prob(23, 365), 3))    # 0.507
print(round(half_threshold(2**16)))         # 301
print(2**53 / 1000 / (365.25*24*3600))     # 285k years
```

— end of document —